Computer Science & Engineering Department

# Arhitectura Sistemelor de Calcul

Curs 5
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest



# In cursul anterior

- De ce avem nevoie de paralelism?
- Care sunt nivelele de prelucrare paralela?
- Clasificarea sistemelor de calcul
- Exemple de utilizare a structurilo: SISD, SIMD, MIMD
- Exemplu cu/fara dependenta de date pe sisteme MIMD
- Limitele scalabilitatii sistemelor paralele de calcul





# Cuprins

- Structura si Setul de Instructiuni SIMD
- Implementarea Salturilor Conditionate in Structuri SIMD
- Organizarea Datelor in Structuri SIMD
- Probleme de Comunicatii intr-un Sistem SIMD
- Deplasarea Ciclica a Datelor intre Procesoare
- Intercalarea Perfecta – Shuffle
- Conectarea Inversa – Inverse Shuffle
- Permutari Elementare

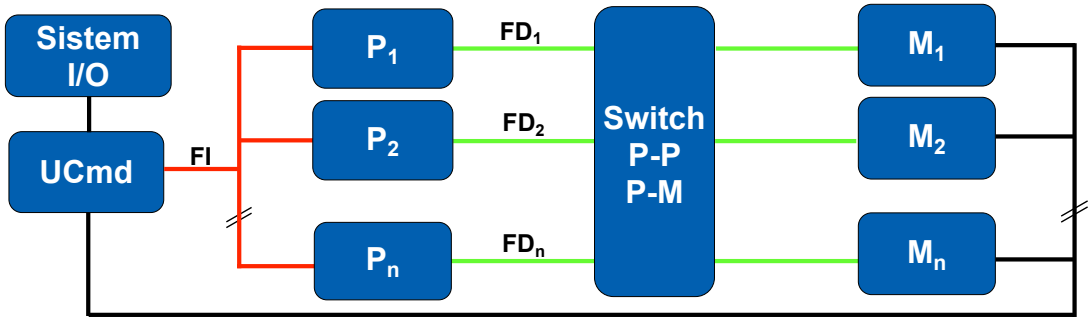





# Structura si Setul de Instructiuni SIMD

# Structura SIMD

- Arhitecturile SIMD sunt adecvate prelucrarilor vectoriale:
  - O singura UCmd
  - Mai multe procesoare aritmetice si module de memorie
  - Retea de comutatie P-P & P-M
- Structuri adecvate prelucrarii vectoriale → esentiala organizarea algoritmului de calcul
- Pentru aplicatii neorientate pe aplicatii vectoriale → ineficiente






# Componentele Structurii SIMD

- **UCmd**: Citeste, interpreteaza si executa instructiuni de control
  - Necesita registre generale, unitate de prelucrare si memorie locala
  - Executa si instructiunile de ramificatie (nu ca $P_i$)
- **$P_i$**: Sunt sincronizate la nivel de instructiune → fiecare procesor lucreaza pe un alt set de date
  - Trebuie sa aiba registre locale
- **M**: Formata din n module
  - Contine instructiuni (de comanda & prelucrare) si date

# Componentele Structurii SIMD

- **Instructiuni**:
  - De control → executate pe UCmd
  - De prelucrare vectoriala → executate de $P_i$
  - Fluxul de instructiuni e similar cu cel din sisteme secventiale
- **Registre index**:
  - Globale → in UCmd; afecteaza cu aceeasi cantitate adresa operanzilor
    - Accesul pe linii
  - Locale → in fiecare Pi; pot fi independente si se refera la zona de memorie Mi asociata fiecarui procesor
    - Acces pe coloane
    - Altfel se poate ajunge la secventialitatea accesului la date!



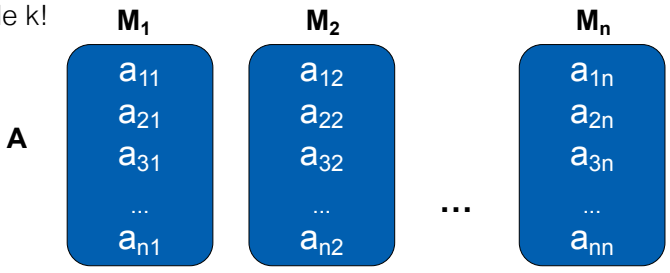


# Exemplu – Inmultirea de Matrice

- Problema: A[n, n], B[n, n], C = A x B
- Considerand ($0 \le k \le n - 1$) → se opereaza pentru toti indicii k simultan, adica se calculeaza pe linii:

```
for j = 0 to n – 1
    cik = cik + aij * bjk  (0 ≤ k ≤ n – 1)
end j loop
```

- Elementele unei linii sunt in module diferite de memorie!
- $a_{ij}$ nu depinde de k!




- Matricele B & C sunt aranjate similar
- Linia rezultat: $c_{i1}, c_{i2} \ldots c_{in}$ este calculata in paralel pe $P_1, P_2 \ldots P_n$





# Setul de Instructiuni Masina SIMD

- $P_i$ are un registru acumulator $ACC_i$ → info locala; UCmd are un registru index INDEX[i]
- Instructiuni masina cu vectori: ($0 \leq k \leq n - 1$)
  - Incarcare:
    - vector: LOAD A ;ACC[k] ← A[0,k] (pentru vectori e suficient A[k])
    - vector indexat: LOAD A[i] ;ACC[k] ← A[INDEX[i],k]
  - Memorare:
    - vector: STO A ;A[0,k] ← ACC[k]
    - vector indexat STO A[i] ;A[INDEX[i],k] ← ACC[k]
  - Adunare vector: ADD A[i] ;ACC[k] ← ACC[k] + A[INDEX[i],k]
  - Inmultire vector: MUL A[i] ;ACC[k] ← ACC[k] * A[INDEX[i],k]
  - Difuzie scalara: BCAST i ;ACC[k] ← ACC[INDEX[i]] (1 perioada de ceas)
- [INDEX[i], k] e organizarea pe linii; [k, INDEX[i]] e organizarea pe coloane
- Instructiunile sunt similare SISD (von Neumman) cu diferenta k-ului ce indica paralelismul





# Instructiuni de Ciclare & Comparare

- Instructiuni referitoare le registre INDEX:
  - Incarcare:
    - ENXC i, ct ;INDEX[i] ← ct
    - LDNX i, Adr ;INDEX[i] ← MEM[Adr]
  - Incrementare:
    - INCX i, ct ;INDEX[i] ← INDEX[i] + ct
  - Compara index & salt la adresa:
    - CPNX i, j, Adr ;if INDEX[i] < INDEX [j] then Jump Adr; else continue;
- Adresele de genul A[i], B[j] sunt adrese indexate cu continutul unui registru INDEX de pe UCmd





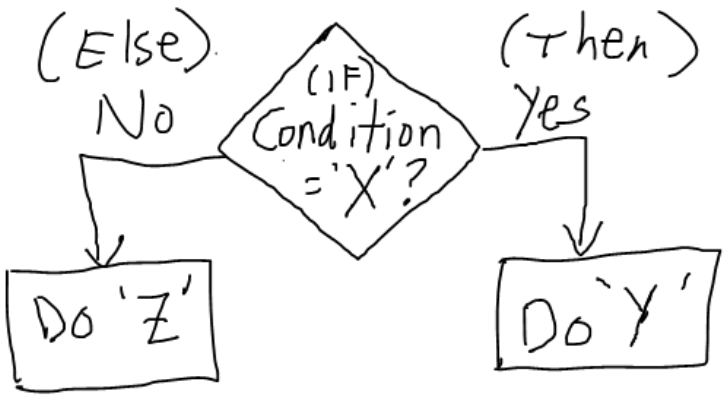
# Exemplu – Inmultirea de Matrice

```
for j = 0 to n – 1
   cik = cik + aij * bjk (0 ≤ k ≤ n – 1)
end j loop
```

- j si n sunt incarcate in doua registre INDEX
- i provine din bucla mare a inmultirii

```
     ENXC j, 0          ;INDEX[j] = 0
     LDNX n, adrN       ;la adrN se afla valoarea n
adr: LOAD A[i]          ;ACC[k] ← A[INDEX[i],k]
     BCAST j            ;ACC[k] ← ACC[INDEX[j]]          (f important! aij)
     MUL B[j]           ;ACC[k] ← ACC[k] * B[INDEX[j],k]
     ADD C[i]           ;ACC[k] ← ACC[k] + C[INDEX[i],k]
     STO C[i]           ;C[i] ← ACC[k]                   (depunere rezultat)
     INCX j, 1          ;INDEX[j]++
     CPNX j, n, adr     ;salt la "adr" cand e cazul
```

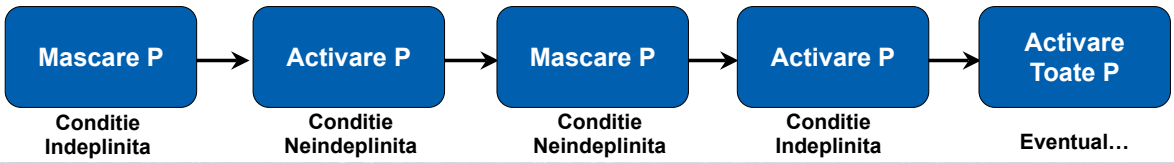





Implementarea Salturilor Conditionate in Structuri SIMD

# Implementarea Salturilor Conditionate

- In sistemele SIMD salturile conditionate se implementeaza prin divizarea fluxului de instructiuni (doar 1FI posibil!)
  - Un flux pentru conditia indeplinita
  - Alt flux pentru conditia neindeplinita
  - Cele doua FI se executa la momente diferite de timp si secvential! → paralelism n/2
  - Pentru implementarea acestor salturi conditionate se vor utiliza operatii de mascare a activitatii procesoarelor $P_i$






# Mascare/Activare Procesoare

- Se considera cate un bit de mascare asociat fiecarui procesor $P_i$, intr-un registru de masca (n biti): $P_i \leftrightarrow$ MASK[i]:
  - MASK[i] = 1 → procesorul $P_i$ functioneaza
  - MASK[i] = 0 → procesorul $P_i$ in asteptare & nu executa operatia curenta
- Operatiile de testare a mastilor se fac in P-uri & rezultatele → Reg Index din UCmd → necesare instructiuni logice & de acces la registre Index:
  - Pentru stabilirea de conditii
  - Pentru implementarea salturilor
  - Lucru cu registre de masti / mascare efectiva

# Instructiuni cu Registre de Masti

- Compara vector:
  - CLSS A, i ;if ACC[k] < A[0,k] then INDEX[i].bitk ← 1
    ;else INDEX[i].bitk ← 0
  - CEQL A, i ;similar doar cu conditia '=' si nu '<'
- Ramificatie fortata:
  - BRALL i, Adr ;Salt la Adr numai daca INDEX[i] = 1
  - BRNON i, Adr ;Salt la Adr numai daca INDEX[i] = 0
- Complementare index:
  - CMP i ;INDEX[i] ← NOT INDEX[i]





# Instructiuni cu Registre de Masti

- Incarca/Memoreaza masca de la index:
  - LDMSK i ;MASK ← INDEX[i]
  - STMSK i ;INDEX[i] ← MASK
- Mentionarea/extragerea primului bit (prioritatea):
  - FRST i ;if INDEX[i] ≠ 0 then ramane nemodificat
  - ;else primul bit 1 ramane 1 si restul de biti 1 se trec pe 0
  - FRST se utilizeaaza cand o parte din Procs indeplinesc o conditie si trebuiesc activate
  - NXFIR i: INDEX[i] = indicele primului bit 1
  - NXFIR se utilizeaza impreuna cu BCAST





# Exemplu – Normalizarea unei Matrice

- Normalizarea prin inlocuirea A[i,j] ← A[i,j]/A[0,j] cu A[0,j] ≠ 0:

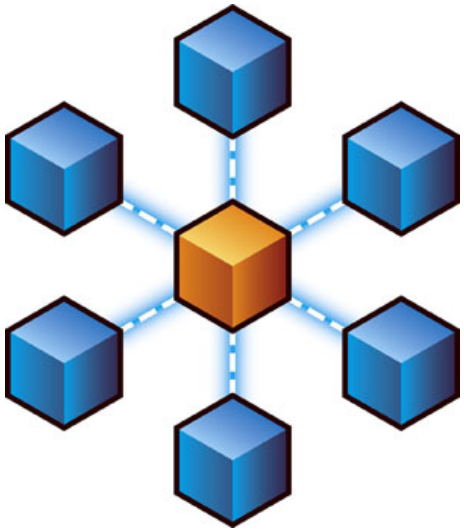
```
for i = 1 to n - 1
   if (A[0,j]!=0) then Aij /= Aoj (0 ≤ j ≤ n – 1)
end i loop
```

```
for i = 1 to n - 1
   Aij /= Aoj, M[j]
end i loop
```

- Fie M[j] = (A[0,j]!=0) in corespondenta cu mastile Pi:

```
      ENXC i, 1          ; INDEX[i] = 1 (contor)
      SUB ACC            ; Resetezi toate ACC-urile la 0 (XOR AX, AX)
      CEQL A, M          ; if ACC[k] = A[0,k] then bit k din INDEX[M] = 1        (0 ≤ k ≤ n – 1)
                         ;   else bit k din INDEX[M] = 0
      CMP M              ; INDEX[M] ← NOT INDEX[M] (setarea mastii pe conditia dorita)
      LDMSK M            ; Incarcam masca & activam procs
      LDNX n, adrN       ; La adrN e valoarea n
adr:  LOAD A[i]          ; ACC[k] ← A[INDEX[i],k]
      DIV A              ; Doar pe procs unde MASK = 1                          (0 ≤ k ≤ n – 1)
      STO A[i]           ; A[i] ← ACC[k] (depunere rezultat)
      INC i, 1           ; INDEX[i]++
      CPNX i, n, adr     ; Salt la “adr” (sfarsitul ciclului)
```





# Organizarea Datelor in Structuri SIMD

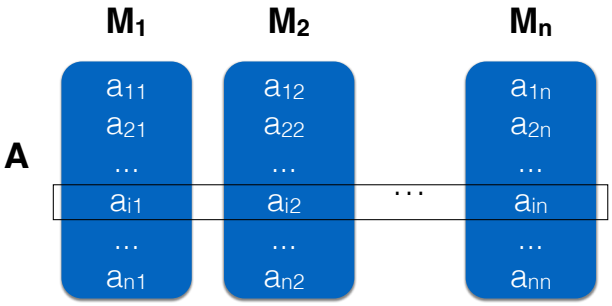
# Organizarea Datelor in Structura SIMD

- Probleme:
  - Repartizarea in memoriile $M_i$ & accesul paralel
  - Citirea vectorilor **intr-o singura instructiune**
  - La matrice: acces la linii si la coloane → necesara transpunerea / deplasarea datelor?
- Memoria:
  - Poate face cel mult un acces la un ciclu de lucru
  - n operanzi in n module diferite pot fi cititi simultan
  - Defavorabila plasarea in acelasi modul al operanzilor → citirea va fi secventiala in n cicli de acces!



# Reprezentarea Directa pe Linii

- Matrice A[n,n] & structura SIMD cu n procs si n memorii
- Cu acces simultan la liniile matricei:
  - Acces paralel la linii
  - Acces secvential la coloane

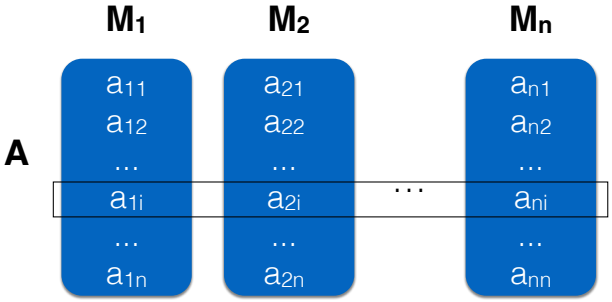



LOAD A[i] – citirea liniei i printr-un singur acces

# Reprezentarea Directa pe Coloane

- Matrice A[n,n] & structura SIMD cu n procs si n memorii
- Cu acces simultan la coloanele matricei:
  - Acces secvential la linii
  - Acces paralel la coloane

Reprezentarile directe pe linii/coloane sunt incompatibile la acelasi moment de timp

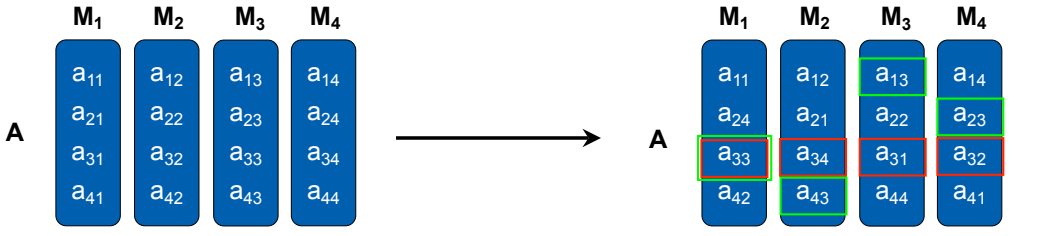



LOAD A[i] – citirea coloanei i printr-un singur acces



# Reprezentarea Deplasata Ciclica - EOL CA

- Matrice A[4,4] & structura SIMD cu 4 procs si 4 memorii
- Cu acces paralel liniile si coloanele matricei:
  - Exemplu valid pentru inmultirea matricelor (linia & coloana 3)




Organizarea originala (pe linii)

Organizarea deplasata

Necesar un shuffling cu reteaua de comutatie ($a_{i1}$ nu e pe $P_1$)

# Concluzii Organizarea Datelor

- Trade-off simplu:
  - Programe simple → acces secvential la linii & coloane
  - Programe complicate → acces paralel la linii & coloane
- Cu organizarea deplasata accesul la linii & coloane se face intr-un singur ciclu:
  - La linii – adresa fizica/actuala a liniei
  - La coloane – sunt necesari registre index private in $P_i$
    - Pt acces la Col i, vectorul index e deplasat ciclic pana 0 e la $M_i$
    - Se difuzeaza adresa liniei 0 cu indicatia de indexare locala
    - Aducerea in pozitia corecta se face prin reteaua de comutatie P-P
- Costul platit paralelismului este **consistenta datelor**!
- Cu reteaua de comutatie (RC) cross-bar → nu conteaza asezarea datelor
  - Se seteaza cross-barul sau indexul local inainte de operatii





# Exemplu – Suma pe Coloane

- Problema: A[n, n] → se doreste suma pe coloane a matricei A intr-un vector SUM[n]
- Cu organizarea pe linii – suma se face secvential, linie cu linie, fiecare $P_i$ calculeaza suma pe cate o coloana:

```
SUM[k] = 0 (0 ≤ k ≤ n - 1)
for i = 0 to n - 1
   LOCINDEX[k] = i (0 ≤ k ≤ n - 1)
   SUM[k] += A[LOCINDEX[k],k] (0 ≤ k ≤ n - 1)
end i loop
```

- LOCINDEX e un index local din P
- Fiecare citire din memorie aduce n elemente ce sunt adunate la cele n sume partiale pe cele n P
- Complexitatea este… $O(n)$

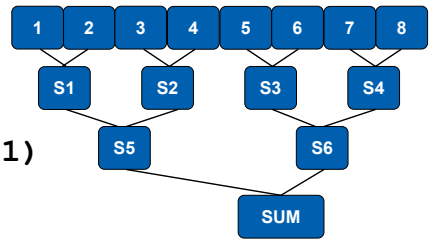




# Exemplu – Suma pe Linii v1

- Problema: A[n, n] → vrem suma pe linii a matricei A in vectorul SUM[n]
  1. Organizarea pe linii duce la o complexitate $O(n^2)$
  2. Organizarea pe coloane duce la un algoritm de transpunere
  3. Folosind registre temporare asociate $P_i$-urilor (TEMP[k]) $O(n \log_2 n)$
  4. Se doreste o complexitate $O(n)$ → trebuie cautata alta solutie
- Solutia propusa este:

```
for i = 0 to n - 1
   TEMP[k] = A[i,k] (0 ≤ k ≤ n - 1)
   for j = 1 to n - 1
      TEMP[k] += TEMP[k-j] (j ≤ k ≤ n - 1)
   end j loop
   SUM[i] = TEMP[n-1]
end i loop
```



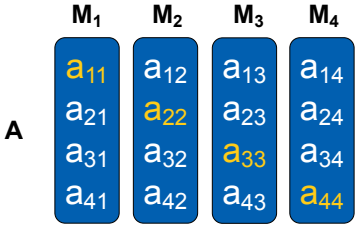

- Complexitatea este… $O(n \log 2n)$





# Exemplu – Suma pe Linii v2

- Solutia pentru o complexitate $O(n)$ → va folosi cate un index local pentru fiecare $P_i$, pentru ca e absolut necesar




- Algoritmul este:

```
SUM[k] = 0 (0 ≤ k ≤ n - 1)
LOCINDEX[k] = k (0 ≤ k ≤ n - 1)
for i = 0 to n - 1
   SUM[k] += A[LOCINDEX[k],(k - i) mod n] (0 ≤ k ≤ n - 1)
end i loop
```

- Se aduna o diagonala la suma curenta
- Diagonala e deplasata ciclic spre elementul adiacent
- A[LOCINDEX[k],k] nu este altceva decat accesul pe coloane!

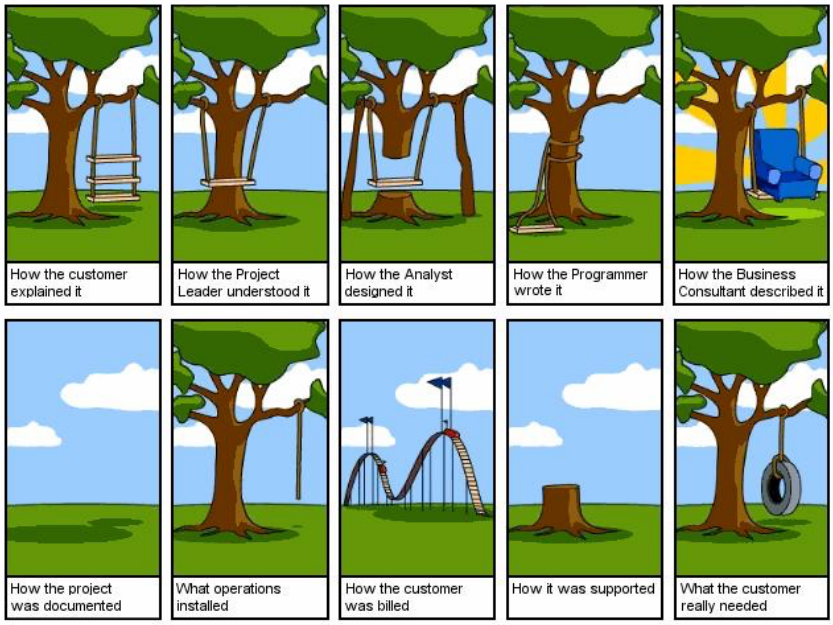




# Concluzii Algoritmi SIMD

- Elaborarea unor algoritmi eficienti necesita
  - Organizarea corespunzatoare a structurilor de date
  - Registre locale ale sistemului de calcul
  - Existenta unei retele de comutatie P-P
- Etapele elaborarii unui program sunt
  - Descrierea algoritmului
  - Paralelizarea algoritmului
  - Optimizarea paralelizarii curente

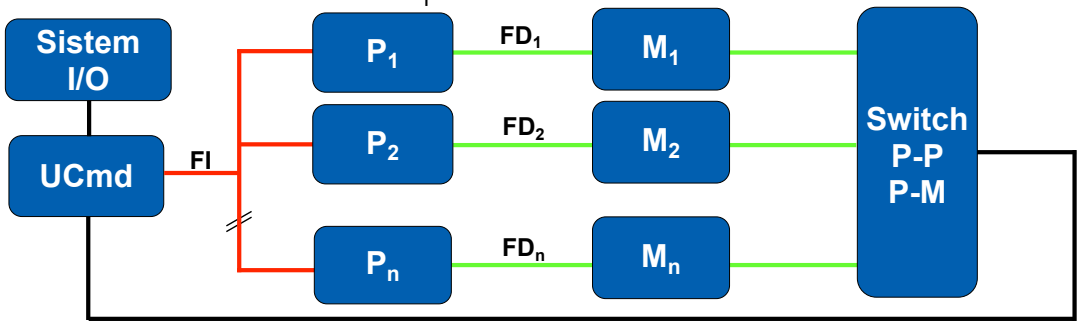





Probleme de Comunicatii intr-un Sistem SIMD

# Probleme in Structura SIMD

- Arhitecturile SIMD sunt adecvate prelucrarilor vectoriale
- Pentru cresterea vitezei masinii SIMD
  - Datele trebuiesc plasate in module distincte de memorie
- Pentru utilizarea eficienta a procesoarelor
  - Datele trebuiesc permutate pentru a furniza operanzi
- Pentru rezolvarea celor doua probleme sunt necesare RC P-P si P-M






# Retele de Comutatie - RC

- RC P-P sunt implementate la nivelul procesoarelor
- RC P-M pot fi implementate cu switch-uri Cross-Bar
  - Pentru n procesoare sunt necesare n(n-1)/2 SP
  - Cand n creste costul devine prohibitiv: 65k Procs → 4Mrd+ Switch-uri
- Sunt necesare RC adecvate fiecarei aplicatii
  - Masini dedicate
- Aceste RC ofera
  - Fiabilitate
  - Costuri reduse
  - Viteza mare de comutatie/transfer
  - Facilitati de intretinere

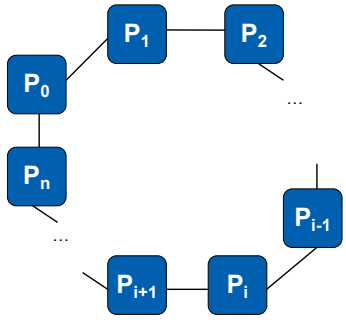
The Trade-off…
Reduced Parallelism!



# Deplasarea Ciclica a Datelor intre Procesoare

# Interconexiunea Circulara

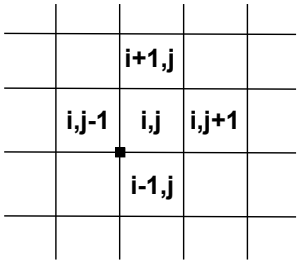
- Deplasarea ciclica a datelor intre procesoare
- Conexiunile sunt permise numai cu vecinii
- Nu exista conexiuni directe intre i si j
- Sunt maxim [(i-j-1)% n] noduri intermediare

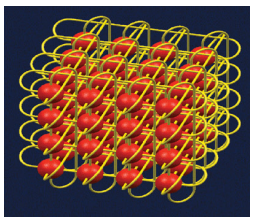


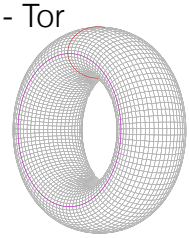

- Exemple de domenii de aplicatie:
  - Adecvate rezolvarii ecuatiilor diferentiale de ordinul 1: x[i] = x[i-1] + x[i+1] – 2x[i]
  - Propagarea caldurii printr-o bara
  - Aplicatii meteo – prognoza vremii in diferite puncte ale globului se face in paralel cu aceleasi date

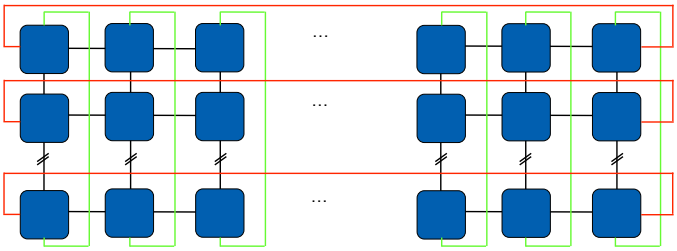
# Interconexiunea Matriceala

- Asigura comunicarea cu patru procesoare vecine
- Daca dependenta este de felul: x[i,j] = x[i+1,j] + x[i-1,j] + x[i,j+1] + x[i,j-1] – 4x[i,j] → de ex: evoluatia temperaturii in volumul unui material
- ILLIAC 4 – primul SIMD (64 procs) pana la BG
  - 90% la fel si acum - Tor

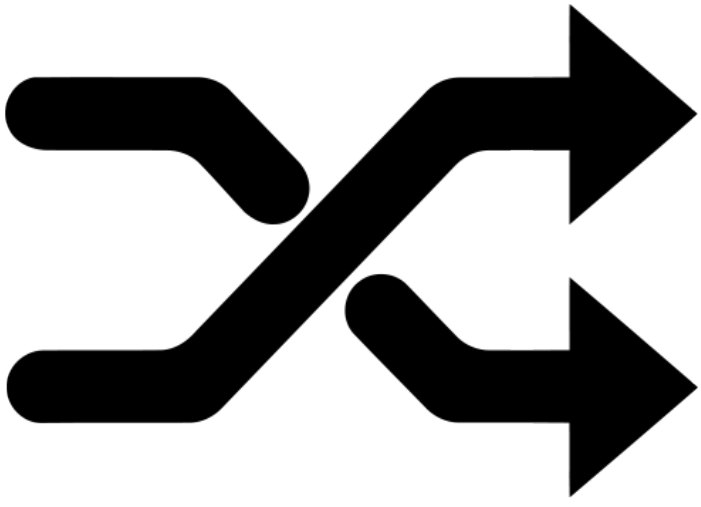






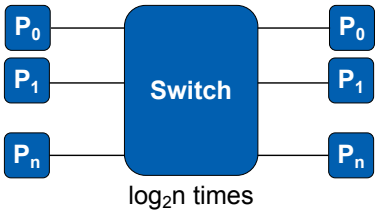




# Intercalarea Perfecta – Shuffle

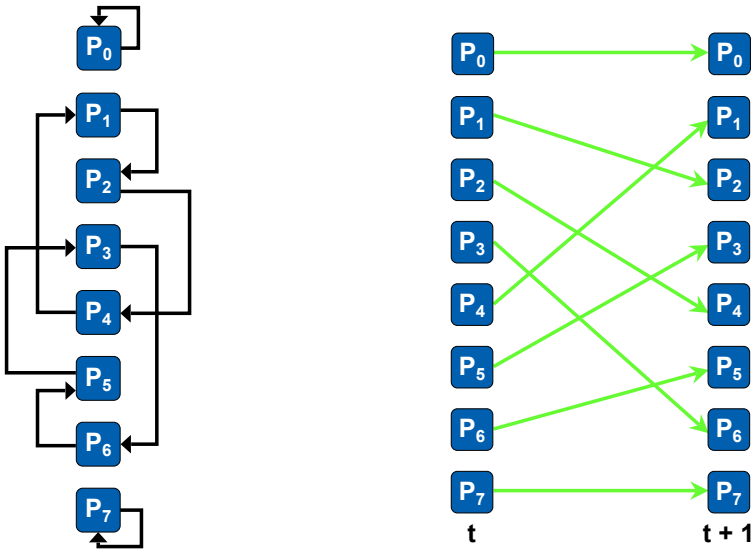
# Intercalarea Perfecta – Shuffle

- Este adecvata pentru:
  - Transformata Fourier (FFT – Fast Fourier Transformation)
  - Algoritmi de sortare
  - Transpuneri de matrice
- Asigura conectarea intre procesoarele unei structuri SIMD de n procesoare cu:
  - n-2 comunicatii simultane
  - dupa log2n iteratii datele ajung pe procesorul sursa

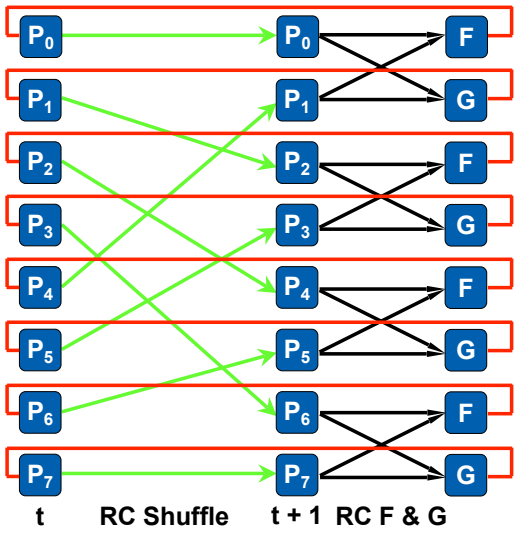





# Shuffle – 8 Procesoare



# Shuffle – 8 Procesoare




- Doua RC:
  - Shuffle
  - Adiacenta (pt F & G):
    - F e pe $Y_i$ = pare
    - G e pe $Y_{i+1}$ = impare
  - $\log_2 n$ iteratii

```
for i = 1 to log2n
   shuffle[y]
   Yi    = F[Yi,Yi+1]
   Yi+1  = G[Yi,Yi+1]
end i loop
```

- Utilizare: sortari, FFT, transpuneri, functii cu operatii recurente





# Shuffle – Aplicatii

- Pentru sortari:
  - F calculeaza maximul celor doi operanzi
  - G calculeaza minimul celor doi operanzi
- Pentru FFT:
  - F produce suma ponderata
  - G produce diferenta ponderata
  - Ponderile sunt calculate dinamic la fiecare pas al iteratiei
- Pentru transpunerea matricelor:
  - Organizarea directa a matricei
  - F = $Y_i$
  - G = $Y_{i+1}$





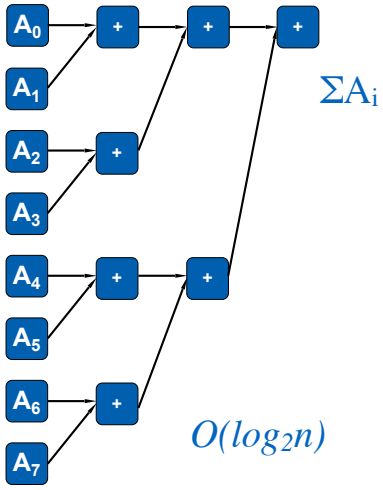
# Shuffle – Aplicatii Iterative

- Suma elementelor unui vector
- Daca vectorul e intr-o singura memorie → secvential:

```
sum = 0
for i = 0 to n-1
    sum += ai
end i loop
```

$O(n)$

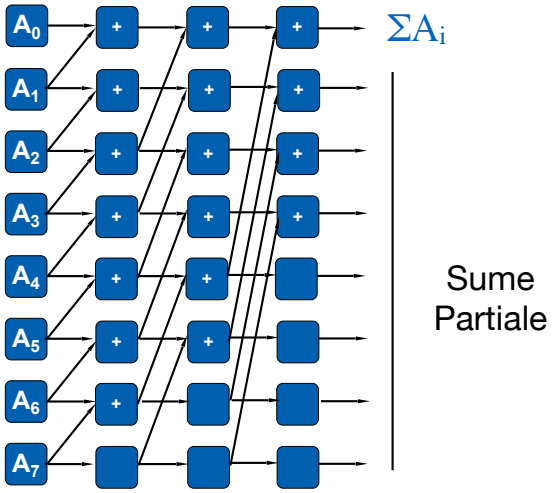
- Se poate reduce $O(n)$ pe SIMD?
- Distribuim vectorul intre memorii
- Probleme:
  - Incarcarea neechilibrata a procs
  - La fiecare pas trebuie schimbata RC in mod dinamic

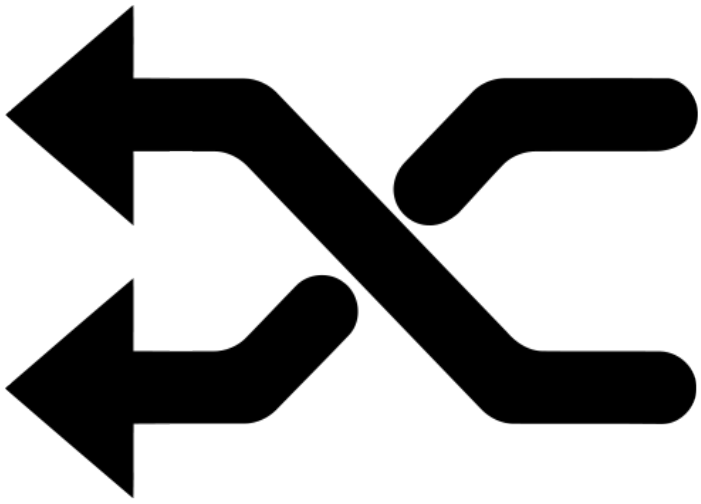





# Shuffle – Aplicatii Iterative

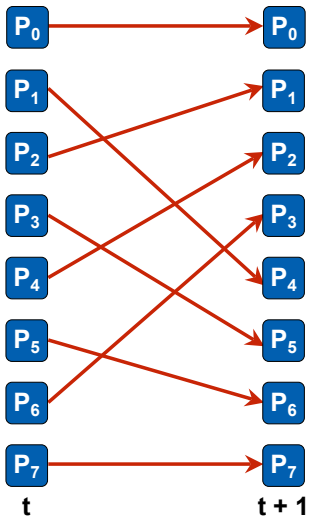
- Sunt necesare
  - Comunicare liniara
  - RC statica
- Reconfigurarea conexiunilor e mai simpla
- Dezavantaj: este in continuare necesara reconfigurarea la fiecare pas!
- Complexitate: $O(log_2 n)$





# Conectarea Inversa – Inverse Shuffle

# Inverse Shuffle – 8 Procesoare

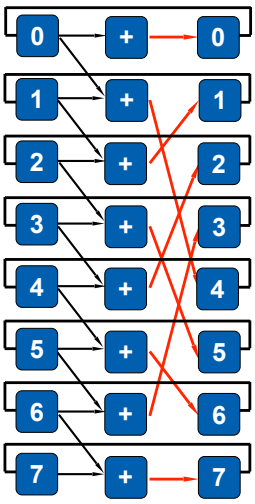



- Adecvat pentru operatii recurente, calculabile in $\log_2 n$ pasi
- Suma elementelor unui vector:

```
Yi = Ai  (0 ≤ i ≤ n-1)
for j = 1 to log2n
   Yj    = Yj + Yj-1
   invshuffle[Y], MASK
   compute MASK
end j loop
```

- Legatura identica pentru toate iteratiile:
  - O legatura directa
  - Un Inverse Shuffle

# Inverse Shuffle – Aplicatii Iterative

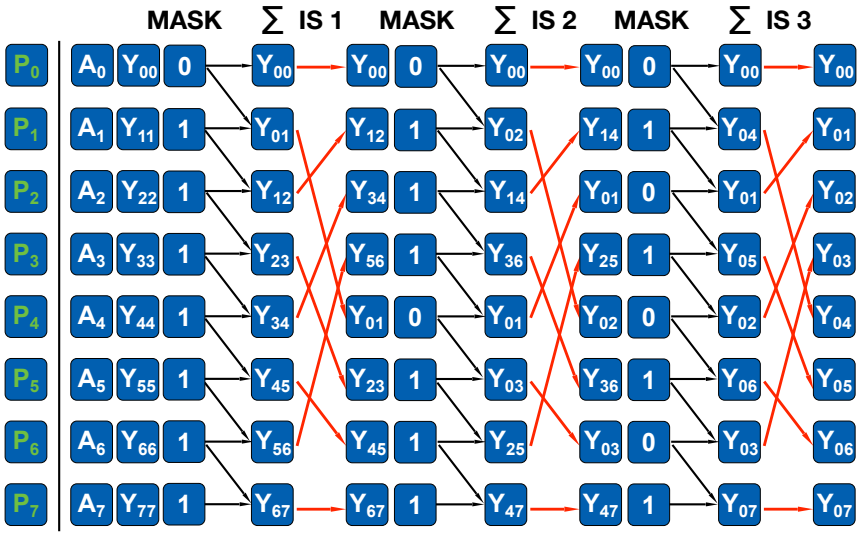



- Legatura (RC) identica pentru toate iteratiile:
  - O legatura directa: $Y_j = Y_j + Y_{j-1}$
  - Un Inverse Shuffle
- Mastile se modifica si ele prin Inverse Shuffle la fiecare iteratie
  - MASK = 0 → procesor inactiv
  - MASK = 1 → procesor activ: (F & G = +)
- La anumite iteratii, unele procesoare pot fi inactivate de masti
- Notam $Y_{ij} = \sum_{k=i}^{j} A(k)$ $O(log_2 n)$





# Inverse Shuffle – RC




$Y_{ij} = \sum_{k=i}^{j} A(k)$





# Concluzii Structuri SIMD

- Probleme adecvate structurilor SIMD:
  - Calculele trebuiesc descrise de iteratii de tip vector a.i. majoritatea timpului sa fie ocupat cu operatii cu caracter identic
  - Operanzii (simultani) trebuiesc repartizati in module diferite de memorie
  - Trebuiesc identificate conexiunile intre operanzi a.i. ele sa fie cat mai aproape de conexiunile fizice intre procesoare
- Clasa problemelor SIMD nu este universala
  - Doar probleme dedicate se preteaza acestor arhitecturi
- Algoritmii, sistemele de interconectare si structura datelor trebuiesc adaptate in consecinta





# Permutari Elementare

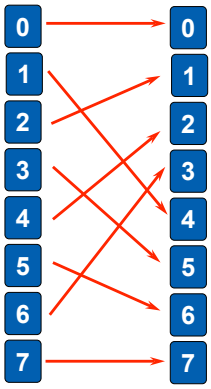
# Permutari Elementare

- Specifica conexiunea intre n resurse ca o functie bijectiva pe o multime ordonata
- Se pot realiza in mai multe feluri:
  1. Permutarea de baza – Base Line Permutation
  2. Permutarea cu intercalare perfecta – Shuffle Permutation
  3. Permutarea cu negare de bit – Negate Bit Permutation
  4. Permutarea fluture – Butterfly Permutation
  5. Permutarea cu ordine inversa – Bit Reversal Permutation
  6. Permutarea cu incrementare modulo – Increment Permutation
- Definirea acestor permutari se poate face pe baza reprezentarii binare a adreselor resurselor din multime (Flanders): ADR = ($a_n$ $a_{n-1}$ … $a_1$ $a_0$)



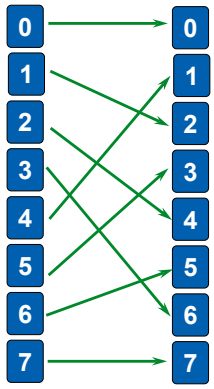
# 1 – Base Line Permutation

- Bitul cel mai nesemnificativ devine cel mai semnificativ:
  - P_base ($a_n$ $a_{n-1}$ … $a_1$ $a_0$) = ($a_0$ $a_n$ $a_{n-1}$ … $a_1$)
- Permutarea de baza este intercalarea perfecta inversa!
- Pentru 8 resurse avem P_base ($a_2$ $a_1$ $a_0$) = ($a_0$ $a_2$ $a_1$)
- Variante posibile sunt:
  - Permutarea de baza k inferioara:
    - Cei mai nesemnificativi k biti din adresa
    - P_b_kinf ($a_n$ … $a_k$ $a_{k-1}$ … $a_1$ $a_0$) = ($a_n$ … $a_k$ $a_0$ $a_{k-1}$ … $a_1$)
    - Asigura conexiunea **in** submultimi de resurse
  - Permutarea de baza k superioara:
    - Cei mai semnificativi k biti din adresa
    - P_b_ksup ($a_n$ … $a_{n-k+1}$ $a_{n-k}$ … $a_0$) = ($a_{n-k+1}$ $a_n$ … $a_{n-k+2}$ $a_{n-k}$ … $a_0$)
    - Asigura conexiunea **intre** submultimi de resurse



# 2 – Shuffle Permutation

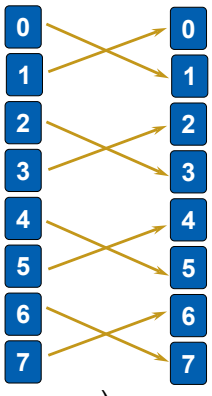
- Bitul cel mai semnificativ devine cel mai nesemnificativ:
  - P_shuffle ($a_n$ $a_{n-1}$ … $a_1$ $a_0$) = ($a_{n-1}$ … $a_1$ $a_0$ $a_n$)
- Pentru 8 resurse avem P_shuffle ($a_2$ $a_1$ $a_0$) = ($a_1$ $a_0$ $a_2$)
- Variante posibile sunt:
  - Permutarea shuffle k inferioara:
    - Cei mai nesemnificativi k biti din adresa
    - P_shuffle_kinf ($a_n$ … $a_k$ $a_{k-1}$ … $a_1$ $a_0$) = ($a_n$ … $a_k$ … $a_1$ $a_0$ $a_{k-1}$)
    - Exemplu: Conexiunea memoriilor - adr baza & offset intr-un modul
  - Permutarea shuffle k superioara:
    - Cei mai semnificativi k biti din adresa
    - P_b_ksup ($a_n$ … $a_{n-k+1}$ $a_{n-k}$ … $a_0$) = ($a_{n-1}$ … $a_{n-k+1}$ $a_n$ $a_{n-k}$ … $a_0$)

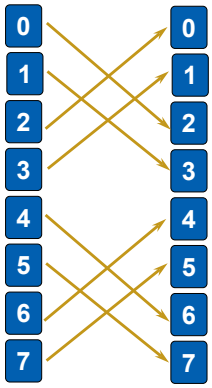





# 3 – Negate Bit Permutation

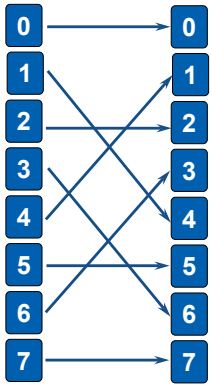
- Bitul cel mai nesemnificativ este negat:
  - P_negate_bit ($a_n$ $a_{n-1}$ … $a_1$ $a_0$) = ($a_n$ $a_{n-1}$ … $a_1$ $\overline{a_0}$)
- Pentru 8 resurse avem:
  - P_negate_bit ($a_2$ $a_1$ $a_0$) = ($a_2$ $a_1$ $\overline{a_0}$)
- Varianta posibila este negarea bitului k:
  - P_negate_bit_k ($a_n$ … $a_{k-1}$ … $a_1$ $a_0$) = ($a_n$ … $\overline{a_{k-1}}$ … $a_1$ $a_0$)
  - Pentru 8 resurse avem P_negate_bit_1 (a2 a1 a0) = (a2 $\overline{a1}$ a0)



# 4 – Butterfly Permutation

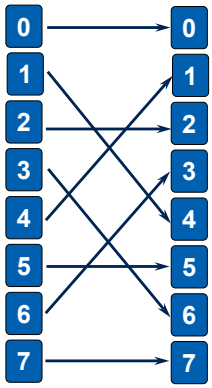
- Se interschimba bitul cel mai semnificativ cu cel mai nesemnificativ:
  - P_butterfly ($a_n \dots a_1\ a_0$) = ($a_0\ a_{n-1} \dots a_1\ a_n$)
  - P_butterfly ($a_2\ a_1\ a_0$) = ($a_0\ a_1\ a_2$)
- Butterfly este o permutare simetrica
- Variante posibile (ofera posibilitatea de conectare intre resursele clasei) sunt:
  - Permutarea butterfly k inferioara:
    - P_butterfly_k_inf ($a_n \dots a_k\ a_{k-1} \dots a_1\ a_0$) = ($a_n \dots a_k\ a_0 \dots a_1\ a_{k-1}$)
  - Permutarea butterfly k superioara:
    - P_butterfly_k_sup ($a_n \dots a_{n-k+1}\ a_{n-k} \dots a_1\ a_0$) = ($a_{n-k+1}\ a_{n-1} \dots a_n\ a_{n-k} \dots a_0$)





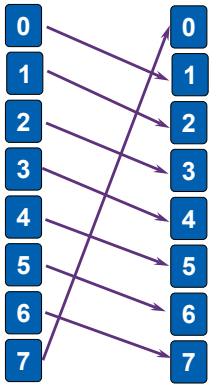

# 5 – Bit Reversal Permutation

- Se interschimba complet ordinea bitilor:
  - P_reversal ($a_n \dots a_1\ a_0$) = ($a_0\ a_1 \dots a_{n-1}\ a_n$)
  - P_reversal ($a_2\ a_1\ a_0$) = ($a_0\ a_1\ a_2$)
- Cu aceasta permutare se pot obtine echivalente
- Variante posibile sunt:
  - Permutarea reversal k inferioara:
    - P_reversal_k_inf ($a_n \dots a_k\ a_{k-1} \dots a_1\ a_0$) = ($a_n \dots a_k\ a_0\ a_1 \dots a_{k-1}$)
  - Permutarea reversal k superioara:
    - P_reversal_k_sup ($a_n \dots a_{n-k+1}\ a_{n-k} \dots a_1\ a_0$) = ($a_{n-k+1} \dots a_n\ a_{n-k} \dots a_0$)



# 6 – Increment Permutation

- Se incrementeaza valorile modulo $2^n$:
  - P_increment (ADR) = $(V_{ADR} + 1) \bmod (2^{n+1})$
- Aceasta este o conexiune inelara intre module alaturate
- 1 din ($V_{ADR}$ + 1), poate fi orice constanta
- Variante posibile sunt:
  - Permutarea increment k inferioara:
    - Realizeaza conexiuni **in cadrul** submultimii de resurse
  - Permutarea increment k superioara:
    - Realizeaza conexiuni **intre** submultimile de resurse



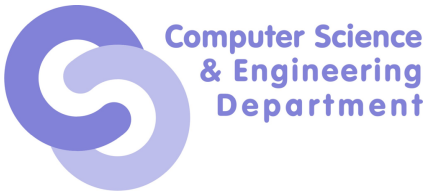



# Ce am invatat in aces curs?

- Structura si Setul de Instructiuni SIMD
- Implementarea Salturilor Conditionate in Structuri SIMD
- Organizarea Datelor in Structuri SIMD
- Probleme de Comunicatii intr-un Sistem SIMD
- Deplasarea Ciclica a Datelor intre Procesoare
- Intercalarea Perfecta – Shuffle
- Conectarea Inversa – Inverse Shuffle
- Permutari Elementare

Thank you for your attention.




emil.slusanschi@cs.pub.ro
cs.pub.ro